

VLM 시스템 구축 매뉴얼

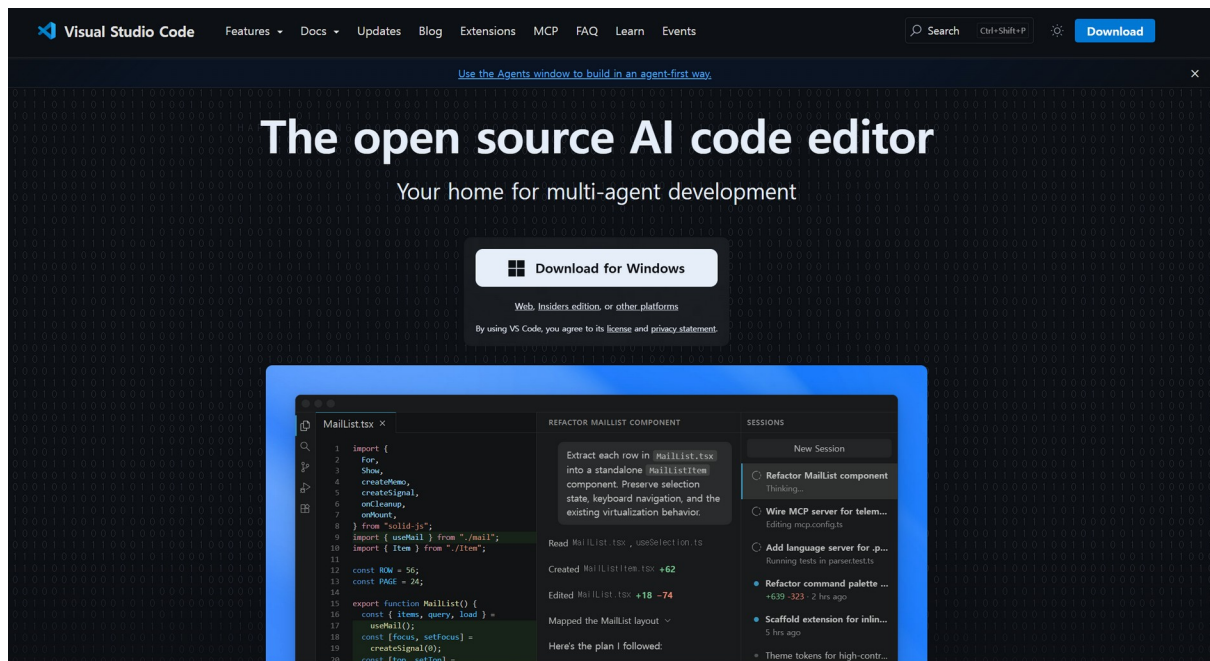
목차

1. 사전 준비
2. 워크스페이스 생성과 VSCode SSH 접속
3. Claude Code 설치
4. Conda 환경 구축
5. 시스템 개요
6. 모델 다운로드
7. 데이터 구성 및 수집
8. 입력 데이터 배치
9. 학습 데이터 확인
10. 예제 코드
11. 추론 실행
12. 파인튜닝
13. 트러블슈팅
14. 부록
15. 부록 — Qwen3.5-9B 대체 백본 (추론 비교)

1. 사전 준비

1-1. VSCode 설치

<https://code.visualstudio.com/> 에서 Windows 용 다운로드 후 기본 옵션으로 설치. 이후 모든 서버 작업은 이 VSCode 를 통해 진행.



1-2. KITECH 플랫폼 계정

1. 플랫폼 URL(<https://ai.bigdata-eng.com/login>) 접속 → 회원가입
2. 로그인 후 워크스페이스 메뉴 접속 및 워크스페이스 생성 클릭
3. 워크스페이스 이름과 용도를 작성하고 하단 자원 및 볼륨(용량) 설정

워크스페이스

워크스페이스 정보

워크스페이스 이름

100자 이하의 워크스페이스 이름

설명

워크스페이스 설명 및 용도

자원

GPU

ON_PREMISE_GPU

NVIDIA-A100-SXM4-40GB

NVIDIA ampere 40.960Mi

VPU 16.0 core, Memory 25.0 Gi

CPU

내 GPU 사용 현황

노드의 할당 가능한 GPU 현황

이 리소스가 할당된 노드가 존재하지 않습니다.

사용중

사용 중인 GPU에 할당

할당됨

대기중



1-3. HuggingFace 계정 · 토큰

1. <https://huggingface.co/join> 에서 가입
2. 로그인 후 <https://huggingface.co/settings/tokens> 접속
3. Create new token → Token type "Read" → 이름 입력 → Create
4. 화면에 표시되는 토큰(hf_...로 시작, 약 40 자)을 Copy 버튼으로 복사해 메모장 등에 보관
(생성 직후 한번만 볼 수 있기 때문에 반드시 기록해두어야 함)

Access Tokens

User Access Tokens

[+ Create new token](#)

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions.

⚠ Do not share your Access Tokens with anyone; we regularly check for leaked Access Tokens and remove them immediately.

< Create new Access Token

Token type

Fine-grained Read Write

⚠ This cannot be changed after token creation.

Token name

Token name

User permissions (yubin11)

Repositories

- ☐ Read access to contents of all repos under your personal namespace
- ☐ View access requests for all gated repos under your personal namespace
- ☐ Read access to contents of all public gated repos you can access
- ☐ Write access to contents/settings of all repos under your personal namespace

Webhooks

- ☐ Access webhooks data
- ☐ Create and manage webhooks

Discussions & Posts

- ☐ Interact with discussions / Open PRs on repos under your personal namespace
- ☐ Interact with discussions / Open PRs on external repos
- ☐ Interact with posts

Inference

- ☐ Make calls to Inference Providers
- ☐ Make calls to your Inference Endpoints ①
- ☐ Manage your Inference Endpoints ①

Collections

- ☐ Read access to all collections under your personal namespace
- ☐ Write access to all collections under your personal namespace

Billing

- ☐ Read access to your billing usage and know if a payment method is set

Jobs

- ☐ Start and manage Jobs on your behalf

ess requests for selected gated repos

Save your Access Token



Save your token value somewhere safe. **You will not be able to see it again after you close this modal.** If you lose it, you'll have to create a new one.

hf_cjusXnBvStHuLXFfDqHI0apeCIvwqTSCFI

 Copy

Name

Permissions

FINEGRAINED

Done

2. 워크스페이스 생성과 VSCode SSH 접속

PC 와 GPU 서버를 SSH 로 연결한다. 워크스페이스는 재시작해도 데이터가 유지된다.

2-1. 워크스페이스 생성

치타 플랫폼 로그인 → 좌측 메뉴 워크스페이스 → 워크스페이스 생성

- 이름: ~~~~
- 자원: "NVIDIA-A100-SXM4-40GB" 카드 클릭 및 노드 개수 할당
- 이미지: 기본값(cheetah/gpu-workspace:ubuntu22.04-cuda12.1-cudnn8)
- 볼륨: 녹색 버튼을 움직여 최소 200 GB 이상 설정

총 1건 (1 / 1)

구동중

yubin

VRAM

40.0 Gi

vCPU

16.0 Core

MEM

251 Gi

리소스구분

GPU

자원

NVIDIA NVIDIA-A100-SXM4-40GB

이미지

cheetah/gpu-workspace:ubuntu22.04-cuda12.1-cudnn8

UPTIME

15일

2-2. 접속 정보 확인 + PEM 다운로드

구동중인 워크스페이스 클릭 후

자원 현황

실행중

NVIDIA-A100-SXM4-40GB

X 1

시작일 2026-04-29 09:24
반납일 -

218.148.55.186

NVIDIA NVIDIA-A100-SXM4-40GB, ampere, ON_PREMISE

VRAM 40.0 Gi

vCPU 16.0 Core

MEM 251 Gi

모니터링

로그

/home/jovyan 경로는 워크스페이스를 재시작해도 복구가능하므로 중요한 데이터는 해당 경로에 두시기 바랍니다.

접속 정보

구분	접속정보
SSH	SSH 접속 명령어 ssh jovyan@218.148.55.186 -p 30628 -i private.pem 사용자 PEM 다운로드 SSH 접속 주소 복사
개발환경	JupyterLab 4.1.4 Notebook 바로가기 JupyterLab 바로가기 접속 주소 복사 접속 토큰 복사
개발환경	VSCode 4.22.0 VSCode 바로가기 접속 토큰 복사
Desktop	XFCE 4.12.0 VNC 바로가기 접속 토큰 복사

사용자 PEM 다운로드 및 접속 IP 와 포트, 접속 명령어 기록



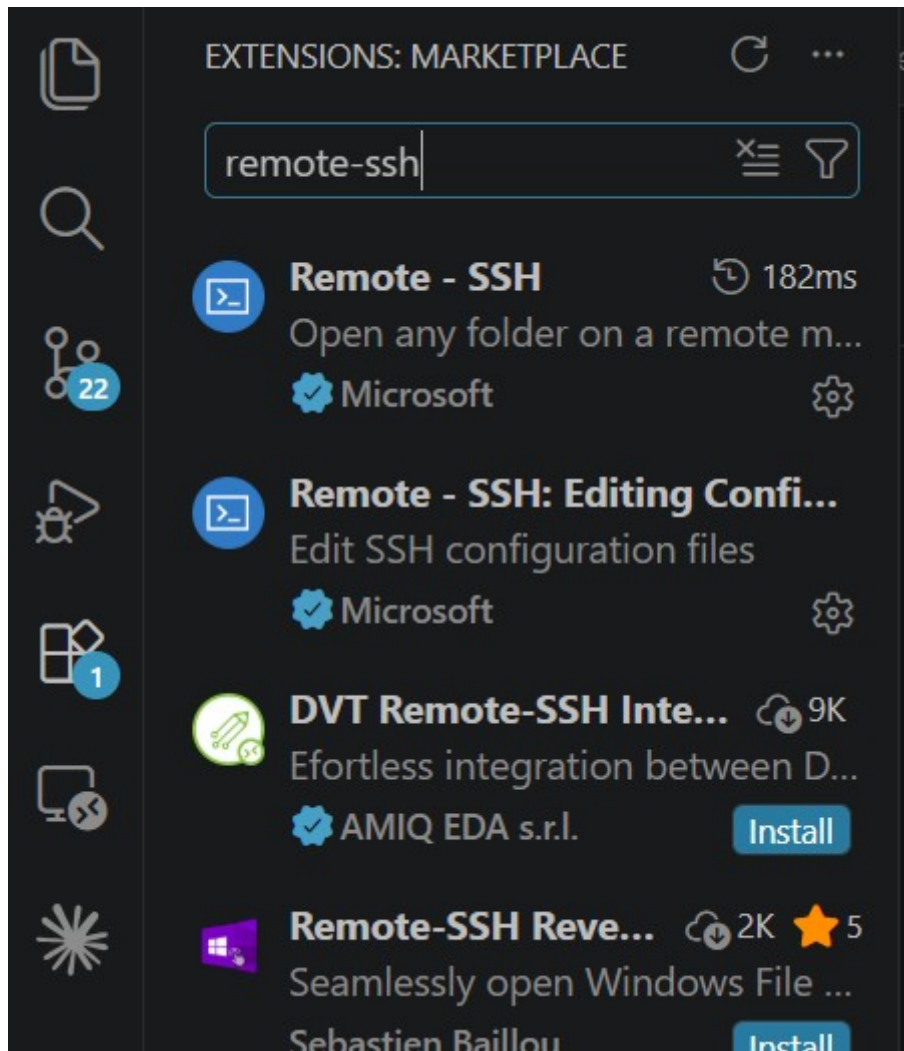
2-3. PEM 키 이동 + 권한 설정

VScode 실행 후 좌측 상단의 Terminal 클릭 → New Terminal 클릭 후 하단에 새롭게 뜬 창에 아래 명령어 입력

```
mkdir $HOME\.ssh -ErrorAction SilentlyContinue
Move-Item $HOME\Downloads\private.pem $HOME\.ssh\
icacls $HOME\.ssh\private.pem /inheritance:r
icacls $HOME\.ssh\private.pem /grant:r "$($env:USERNAME):(R)"
```

2-4. Remote-SSH 확장 설치

VScode 실행 → 좌측 메뉴의 네번째 항목 Extensions 선택 → "Remote - SSH" 검색 → Microsoft 가 게시한 항목 Install



2-5. SSH config 등록

1. VSCode 좌측 상단 View → Command Palette 클릭
2. 입력창에 "Remote-SSH: Open SSH Configuration File" 입력 후 선택
3. 파일 목록이 뜨면 첫 번째 항목(C:\Users\사용자명.ssh\config 경로) 선택
4. 열린 파일에 다음과 같은 형태로 추가

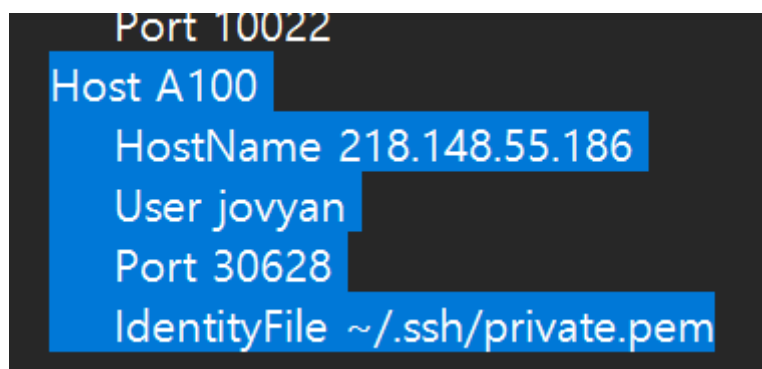
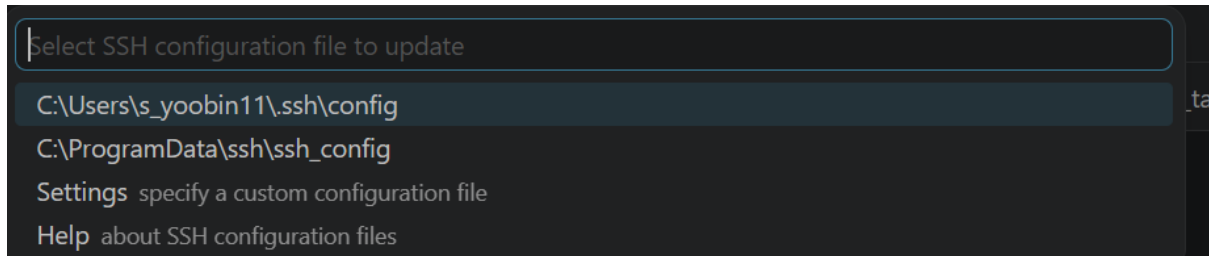
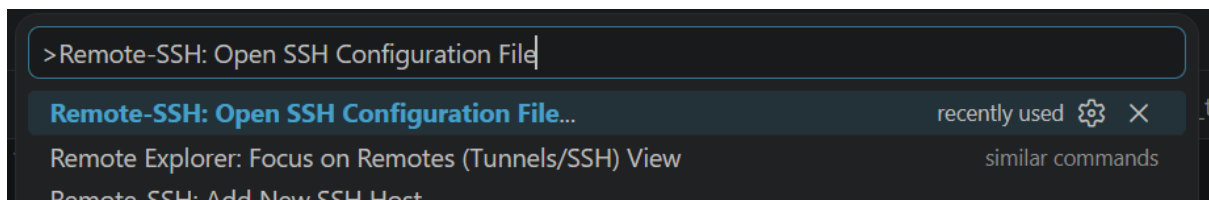
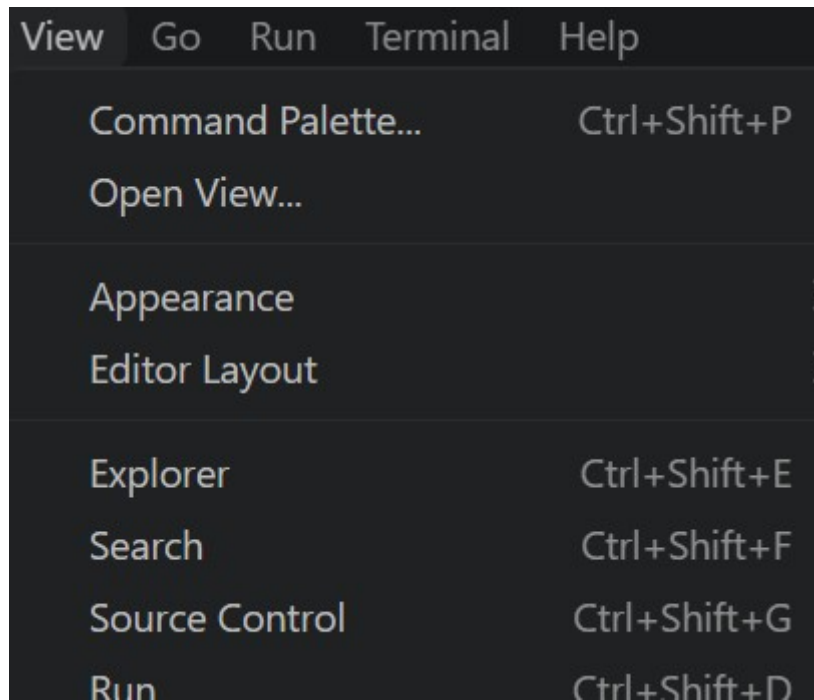
Host 별칭

HostName ip 주소

User jovyan

Port 포트번호

IdentityFile ~/.ssh/private.pem



이렇게 등록해두면 매번 IP, 포트, 키 경로를 다 적지 않고 설정해둔 별칭만 입력해 간편하게 접속할 수 있다. Ex) ssh A100

```
(base) PS C:\Users\s_yoobin11> ssh A100
*****
*                               *
*      Welcome to Cheetah!      *
*                               *
*  This system is for the use of authorized users only. Usage of    *
*  this system may be monitored and recorded by system personnel.  *
*                               *
*  Anyone using this system expressly consents to such monitoring   *
*  and is advised that if such monitoring reveals possible         *
*  evidence of criminal activity, system personnel may provide the  *
*  evidence from such monitoring to law enforcement officials.      *
*                               *
*  Disconnect IMMEDIATELY if you are not an authorized user!       *
*                               *
*****

Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.4.0-139-generic x86_64)

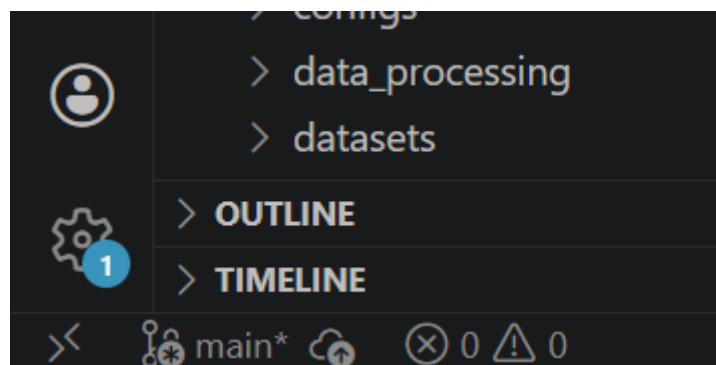
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

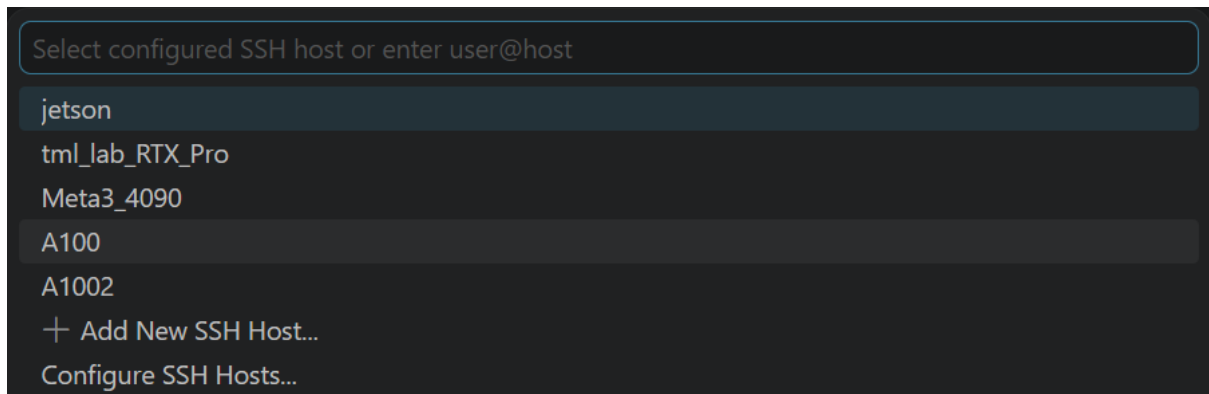
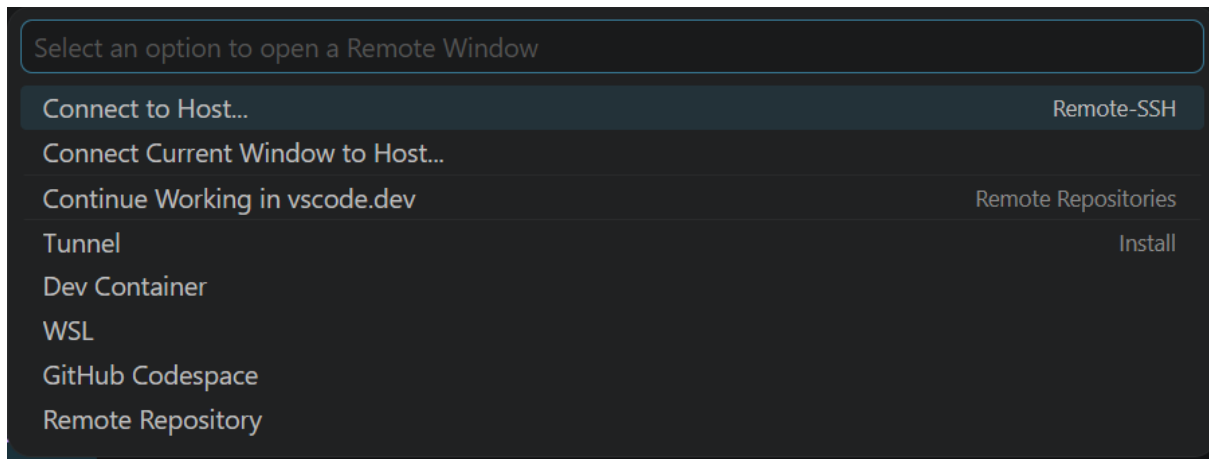
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
```

2-6. 접속

1. VSCode 좌측 하단 >< 모양 파란 버튼 클릭
2. "Connect to Host..." → 별칭 선택
3. 새 VSCode 창에서 fingerprint 확인 메시지가 뜨면 Continue 혹은 Yes 입력
4. 좌측 하단 표시가 "SSH: 별칭"으로 바뀌면 성공





이후 운영체제 선택에서 ubuntu 를 선택한 후 진행한다.



2-7. 접속 후 — 사양 확인과 작업 폴더

Terminal → New Terminal 로 아래 명령어들을 입력해 우측 #과 같은 사양이 나오는지 확인한다.

```
pwd                # /home/jovyan
whoami             # jovyan
nvidia-smi         # A100-SXM4-40GB, CUDA 12.x
```

```
jovyan@cheetah-796f6f62696e3131-6uly1d-6cf846895c-2mnnd:~$ nvidia-smi
Thu May 14 06:40:20 2026

+-----+
| NVIDIA-SMI 550.127.08                  Driver Version: 550.127.08      CUDA Version: 12.4      |
+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====  
|  0   NVIDIA A100-SXM4-40GB         Off | 00000000:01:00.0 Off |             0        |
| N/A   39C    P0               107W / 400W | 33893MiB / 40960MiB |      0%      Default |
|                                     |                      | Default      Disabled |
+-----+-----+-----+

+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|      ID    ID                                   |             Usage |
|=====|
+-----+
```

작업 폴더 생성

```
cd
mkdir -p vlm_poc
```

VSCode 메뉴 File → Open Folder → /home/jovyan/vlm_poc → OK. 로 좌측에 파일 구조와 각 항목에 접근할 수 있다.

3. Claude Code 설치

VSCode 에서 자연어로 코드를 작성, 수정할 수 있는 claude 로 원하는 시스템에 대한 구체적인 설명을 통해 시스템 구축 및 내부 코드 작성, 오류 해결 등의 용도로 쓴다.

SSH 접속 상태에서는 확장이 서버 쪽에 설치되어 서버의 Node.js 위에서 동작한다. 빌트인 우분투 이미지에는 Node.js 가 없으므로 먼저 설치해야 한다.

3-1. 서버에 Node.js 설치

VSCode 서버 터미널에서:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt install -y nodejs  
node --version      # v20.x.x  
npm --version
```

```
jovyan@cheetah-796f6f62696e3131-6uly1d-6cf846895c-2mnnd:~$ node --version      # v20.x.x  
v20.13.1  
jovyan@cheetah-796f6f62696e3131-6uly1d-6cf846895c-2mnnd:~$ npm --version  
10.8.0
```

3-2. 설치

서버 터미널에서 다음을 실행.

```
npm install -g @anthropic-ai/claude-code
```

설치 확인:

```
claude --version
```

버전 번호가 출력되면 OK.

3-3. 로그인

```
cd /home/jovyan/vlm_poc  
claude
```

```
# 이후에 시작할 때는  
claude --resume
```

처음 실행하면 인증 안내가 뜬다.

1. 터미널에 표시되는 URL 을 복사 혹은 ctrl 후 링크를 누르면 자동 연결
2. 본인 PC 브라우저에서 Claude 계정으로 로그인
3. 인증 코드 표시 → 복사

4. 서버 터미널로 돌아와 인증 코드 붙여넣고 엔터

3-4. 동작 확인

채팅창에 입력 후 엔터.

안녕? 너 누구야?

자기소개 응답이 오면 설치 완료.

4. Conda 환경 구축

4-0. Conda 란 무엇이고 왜 쓰는가

Conda 는 같은 컴퓨터 안에 서로 다른 Python·라이브러리 조합을 여러 개 만들어두고 골라 쓸 수 있게 해주는 도구다. 각각의 조합을 "환경(environment)"이라고 부른다.

각각의 환경마다 파이썬 라이브러리들은 서로 의존성이 얹혀 있어서, A 프로젝트는 transformers 4.x 를 요구하는데 B 프로젝트는 5.x 를 요구하는 식의 충돌이 흔하다. 기본 시스템에 프로젝트를 진행할때마다 하나씩 설치하면 다른쪽이 깨진다. Conda 환경은 폴더 단위로 격리되어 있어서, conda activate <이름>으로 들어가면 그 환경의 라이브러리만 보이고, 빠져나오면 깔끔히 분리된다.

환경	용도
vlm_poc	학습·추론 클라이언트
vllm	vLLM 서빙 전용

4-1. Miniconda 설치

```
cd ~
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p $HOME/miniconda3
source ~/miniconda3/etc/profile.d/conda.sh
conda init bash
exec bash
```

4-2. vlm_poc 환경

```
conda create -n vlm_poc python=3.11 -y
conda activate vlm_poc

pip install torch==2.5.1 torchvision==0.20.1 --index-url
https://download.pytorch.org/whl/cu121

pip install "transformers>=5.7" "accelerate>=1.13" "qwen-vl-
utils>=0.0.14" \
    "peft>=0.19" "trl>=1.3" "bitsandbytes>=0.49" "datasets>=4.8" \
    "openai>=2.0" "pdf2image>=1.17" Pillow "jsonschema>=4.26"

conda install -c conda-forge poppler -y
```

4-3. vllm 환경

```
conda deactivate
conda create -n vllm python=3.11 -y
conda activate vllm
pip install vllm
```

pip install vllm 한 줄이 transformers·torch 까지 자동으로 끌어온다.

4-4. 검증

```
conda activate vlm_poc
python -c "import torch, transformers, trl, peft, openai; print('torch:',
torch.__version__); print('cuda:', torch.cuda.is_available()); print('gpu:',
torch.cuda.get_device_name(0))"
```

기대 출력:

```
torch: 2.5.1+cu121
cuda: True
gpu: NVIDIA A100-SXM4-40GB.
```


5. 시스템 개요

5-1. 하는 일

화학물질 안전문서(PDF·라벨 사진)를 입력으로 받아 7 개 항목을 JSON 으로 반환.

항목	예시
product_name	"Acetone"
manufacturer	"Sigma-Aldrich"
cas_number	"67-64-1"
ghs_classification	픽토그램·신호어·H 문구
flash_point_celsius	-20°C (closed cup)
boiling_point_celsius	56°C
first_aid_measures	흡입·피부·눈·삼킴 각 경로별 조치

5-2. 진행 흐름

```
원본 문서 (PDF, JPG)
  ↓ preprocess (페이지 이미지 변환)
manifest + pages/*.png
  ↓ extract (vLLM 호출 + JSON 파싱 + 스키마 검증)
extraction.json + cot.log
```

5-3. 사용하는 도구

백본 모델은 Qwen3-VL-8B-Instruct. Alibaba 오픈소스 VLM(Vision-Language Model)로 이미지·텍스트를 동시에 이해한다. 한국어·영어·중국어 모두 능숙. 파라미터 80 억, 파일 크기 약 17.5GB, Apache 2.0 라이선스. HuggingFace 에서 받는다.

추론 서버는 vLLM. UC Berkeley 에서 만든 LLM 추론 서버로 모델을 메모리에 띄워두고 HTTP API 로 받는다. OpenAI API 호환.

파인튜닝은 LoRA(Low-Rank Adaptation). 17GB 본체를 통째로 학습하지 않고 필요한 부분만 추가 학습

6. 모델 다운로드

6-1. Hugging Face 로그인

```
conda activate vlm_poc  
hf auth login
```

"Add token as git credential? (Y/n)"은 그냥 엔터(기본 Y).

6-2. 다운로드

```
hf download Qwen/Qwen3-VL-8B-Instruct
```

6-3. 확인

```
du -sh ~/.cache/huggingface/hub/models--Qwen--Qwen3-VL-8B-Instruct/
```

7. 데이터 수집

이 시스템은 화학물질 안전문서(MSDS·SDS·제품 라벨·TDS)를 입력으로 받는다. 이 장은 무엇을,

어디서, 어떻게 모으는지를 다룬다.

7-1. 무엇을 수집하나

수집의 기본 단위는 **한 화학물질** 이다. 한 화학물질에 대해 제품 라벨 사진 + 한국어 MSDS + 영문 SDS 를 한 묶음으로 모으는 것을 권장한다. 같은 값이 여러 문서에서 확인될수록 추출 신뢰도가 올라가기 때문이다.

문서 종류	내용
MSDS / SDS	물질안전보건자료. 16 개 표준 섹션으로 구성된다(제품 정보, 유해성, 응급조치, 물리·화학적 특성 등). 한국어 MSDS 와 영문 SDS 를 함께 모은다. 보통 PDF.
제품 라벨	용기에 부착된 라벨. GHS 픽토그램·신호어·H 문구·CAS 번호가 압축적으로 표기되어 있다. 실물을 촬영한 이미지(JPG/PNG).
TDS	기술자료서. 물리·화학적 특성 위주의 보조 자료. 선택 사항.

7-2. 어디서 구하나

- **한국어 MSDS** — KOSHA(한국산업안전보건공단)의 MSDS 검색 시스템에서 받는다. 화학물질명 또는 CAS 번호로 조회한다.
- **영문 SDS** — 화학물질 제조사·공급사(Sigma-Aldrich, Merck, Fisher, Honeywell 등)의 제품 페이지에서 SDS 를 내려받는다.
- **제품 라벨** — 실물 용기의 라벨을 직접 촬영한다. 자료 출처가 사내 보유 화학물질이라면 현장에서 촬영하는 것이 가장 정확하다.

7-3. 수집 시 유의사항

- **형식** — MSDS·SDS 는 PDF, 라벨은 이미지(JPG·PNG)가 일반적이다. 두 형식 모두 파이프라인이 처리한다.
- **화질** — 라벨 사진은 글자와 픽토그램이 또렷하게 보여야 한다. 빛 반사·심한 기울기·접힘이 있으면 추출 정확도가 떨어진다.
- **파일명 규칙** — <화학물질>_<종류>.<확장자> 형식으로 둔다(예: methanol_label.jpg, methanol_msds_ko.pdf, methanol_sds_en.pdf). 파일명에 label·msds·sds·tds 키워드가 들어 있어야 문서 종류가 자동 인식된다.

8. 입력 데이터 배치

8-1. 추론 입력 파일



위 파일을 다운받아 서버에 연결한 VSCode 의 좌측 사이드바에 드래그해 넣으면 서버에 업로드 된다.

8-2. 압축 해제

서버 터미널에서

```
cd /home/jovyan
unzip vlm_poc.zip
ls vlm_poc/
```

를 진행해 압축을 해제한다

8-3. 추론 입력 파일 확인

vlm_poc/inputs/에 4 개 파일이 들어 있다. 추론 실행 시 이 4 건의 문서를 입력으로 진행되어 출력한다.

파일	내용
acetone_label.jpg	아세톤 용기 라벨 사진
acetone_msds_ko.pdf	한국어 MSDS 13 페이지
acetone_sds_en.pdf	영문 SDS 10 페이지
dowanol_pm_tds_en.pdf	영문 TDS 2 페이지

9. 학습 데이터 확인

8장에서 vlm_poc.zip 을 풀 때 학습 데이터(ft_data_pack)도 함께 풀린다.

경로: vlm_poc/data/ft_data_pack/

9-1. 폴더 구조

```
vlm_poc/data/ft_data_pack/  
├─ inputs/           원본 PDF/이미지 15 건  
├─ pages/           PDF 페이지 PNG 변환본  
├─ train_set/       학습용 페어 5 건  
├─ ground_truth/   사람이 검수한 정답 5 건  
└─ README.md
```

9-2. 정답 데이터

VSCode 사이드바: vlm_poc/data/ft_data_pack/ground_truth → acetone.json 더블클릭

```
1  {  
2    "product_name": {  
3      "value": "ACETONE",  
4      "source": {  
5        "filename": "acetone_label.jpg",  
6        "page": 1,  
7        "section": "product_name"  
8      },  
9      "confidence": 0.97  
10   },  
11   "manufacturer": {  
12     "value": "SK CHEMICALS DAEJUNG CO., LTD.",  
13     "source": {  
14       "filename": "acetone_label.jpg",  
15       "page": 1,  
16       "section": "supplier"  
17     },  
18     "confidence": 0.95  
19   },  
20   "cas_number": {  
21     "value": "67-64-1",  
22     "source": {  
23       "filename": "acetone_label.jpg",  
24       "page": 1,  
25       "section": "product_identifiers"  
26     },  
27     "confidence": 0.98  
28   },  
29   "ghs_classification": {  
30     "pictograms": [  
31       "GHS02",  
32       "GHS07"  
33     ],  
34     "signal_word": "Danger",  
35     "h_codes": [  
36       "H225",  
37       "H319",  
38       "H336"  
39     ],  
40   }
```

사람이 검수한 정답 데이터로 파인튜닝 시에 모델 출력의 목표가 되는 학습 타겟이다.

9-3. 데이터 검증

정답을 손으로 작성하면 심표 누락·따옴표 오류·스키마에 없는 키 같은 실수가 섞이기 쉽고, 이런 데이터는 학습을 조용히 망가뜨린다. 학습에 넣기 전에 아래 스크립트로 데이터 묶음을 한 번에

검사한다. 데모 데이터(ft_data_pack)와 12 장에서 직접 만든 데이터 모두 같은 방법으로 확인한다.

검사 항목은 네 가지다.

- **정답 JSON 문법** — 모든 ground_truth/<chem>.json 이 깨지지 않고 파싱되는지.
- **스키마 적합성** — extraction_schema.json 기준으로 7 개 항목·타입·required·GHS enum 이 맞는지.
- **이미지 경로** — train_set/<chem>.json 의 image_paths 가 실제로 존재하는 PNG 를 가리키는지.
- **응답 본문** — train_set 의 response 에서 </thinking> 뒤 JSON 이 파싱되는지.

vlm_poc 디렉터리에서 실행한다.

```
conda activate vlm_poc
cd /home/jovyan/vlm_poc
python <<'PY'
import json, glob, os
from jsonschema import Draft202012Validator

PACK = "data/ft_data_pack"
SCHEMA = "schema/extraction_schema.json"
validator = Draft202012Validator(json.load(open(SCHEMA, encoding="utf-8")))

errors = 0
gt = sorted(glob.glob(f"{PACK}/ground_truth/*.json"))
print(f"정답 파일 {len(gt)}건 검사\n")

for path in gt:
    name = os.path.basename(path)
    try:
        data = json.load(open(path, encoding="utf-8"))          # (1) 문법
    except json.JSONDecodeError as e:
        print(f"[JSON] {name}: {e}"); errors += 1; continue
    for err in validator.iter_errors(data):                      # (2) 스키마
        loc = "/".join(map(str, err.path)) or "(root)"
        print(f"[스키마] {name} @ {loc}: {err.message}"); errors += 1

for path in sorted(glob.glob(f"{PACK}/train_set/*.json")):
    name = os.path.basename(path)
    s = json.load(open(path, encoding="utf-8"))
    for img in s.get("image_paths", []):                        # (3) 이미지 경로
        if not os.path.exists(os.path.join(PACK, img)):
            print(f"[이미지] {name}: {img} 없음"); errors += 1
    body = s.get("response", "").split("</thinking>")[-1].strip()
    try:
        json.loads(body)                                         # (4) 응답 본문
    except json.JSONDecodeError as e:
        print(f"[응답] {name}: {e}"); errors += 1
```

```
print("\n검증 통과 — 문제 없음." if errors == 0 else f"\n총 {errors}건 — 위 항목을  
고쳐야 한다.")  
PY
```

정상이면 다음과 같이 나온다.

정답 파일 5 건 검사

검증 통과 — 문제 없음.

문제가 있으면 파일명과 위치가 출력되므로 해당 항목만 고쳐 다시 돌린다. **'검증 통과'가 나와야 학습으로 넘어간다.**

10. 예제 코드

8 장에서 본 vlm_poc/ 폴더의 구조와 주요 스크립트.

10-1. 폴더 구조

```
vlm_poc/
├─ README.md
├─ requirements.txt
├─ inputs/                ← 추론 입력 (PDF, JPG)
├─ prompts/              ← system_prompt + user_prompt_template
├─ schema/extraction_schema.json ← 출력 형식 검증
├─ scripts/
│   ├─ preprocess.py      ← PDF/JPG → 페이지 PNG
│   ├─ extract.py         ← vLLM 호출 + JSON 파싱
│   ├─ finetune_lora_v2.py ← LoRA 학습
│   ├─ merge_lora.py       ← 어댑터를 base 모델에 머지
│   └─ evaluate.py        ← 정답과 비교 (선택)
├─ data/ft_data_pack/    ← 학습 데이터
├─ outputs/              ← 추론 결과
├─ checkpoints/          ← 학습 산출물
└─ logs/
```

10-2. 주요 스크립트

스크립트	역할
preprocess.py	원본 PDF 를 페이지 단위 PNG 로 변환하고 manifest 생성
extract.py	vLLM 서버에 이미지+프롬프트 전송, 응답 JSON 파싱, 스키마 검증
merge_lora.py	학습된 LoRA 어댑터를 base 모델에 머지해 단독 모델 생성

10-3. 프롬프트와 스키마

모델에 보내는 입력을 “프롬프트”라 부른다. 프롬프트는 시스템 프롬프트와 사용자 프롬프트 두 가지로 구성된다.

시스템 프롬프트는 모델의 역할과 규칙을 정의하는 고정 지시문이다. 모든 호출에 매번 동일하게 들어간다. 예: "너는 화학물질 안전데이터 추출 전문가다. 응답은 반드시 정해진 JSON 형식만 사용해라." 한 번 정해 두면 매번 똑같이 보내므로 모델의 역할과 성격을 고정하는 역할을 한다.

사용자 프롬프트는 호출마다 달라지는 작업 지시로 이번에 처리할 문서, 추출할 항목 등 변하는 부분이 들어간다.

매 입력은 시스템 프롬프트 + 사용자 프롬프트 + 입력 이미지를 모델에 함께 던지는 것이고, 모델은 셋을 합쳐 보고 응답한다.

prompts/system_prompt.txt — 모델에게 다음 세 갈래의 규칙을 강제한다(세부 6 개 규칙과 작성법은 12-4 참고).

- 응답 형식: <thinking>...</thinking> 추론 블록 + JSON 한 객체. 스키마에 정의된 키만 사용하고 마크다운 코드펜스는 쓰지 않는다.

- 환각 방지: 이미지에서 직접 읽을 수 있는 값만 사용한다(외부 지식 금지).

모든 값에 출처와 신뢰도를 명시하고, 둘 이상의 문서에서 같은 값이 확인되면 corroborated_by 에 추가 출처를 적고 confidence 를 0.95 이상으로 올린다.

- 정규화: 온도는 °C 통일, CAS 번호는 'NNNN-NN-N' 하이픈 형식, GHS 픽토그램은 GHS01~GHS09 코드로 매핑, 응급조치는 한 줄로 압축.

<thinking> 블록은 "어느 첨부 어느 섹션을 봤기에 그 값을 골랐는지" 모델의 사고 과정이 담기고, 추출 후 cot.log 로 분리 저장된다.

prompts/user_prompt_template.txt — 매 호출마다 채워 보내는 양식.

추출 대상 7 개 필드(제품명·제조사·CAS 번호·GHS 분류·인화점·끓는점·응급조치), 입력 파일 매니페스트, 스키마 본문이 들어간다. 시스템 프롬프트가 "어떻게" 답할지를 정의한다면, 사용자 프롬프트는 "무엇을" 추출할지를 정의한다.

schema/extraction_schema.json 은 출력 JSON 이 가져야 할 정확한 모양을 정의한 JSON Schema 파일이다. 필드 이름, 데이터 타입, 필수 여부, 출처 표기 구조까지 모두 명시되어 있다.

추출 스크립트(extract.py)는 모델 응답을 받으면 다음 순서로 동작한다.

1. 응답에서 JSON 부분만 분리해 파싱
2. extraction_schema.json 기준으로 검증
3. 위반사항이 있으면 schema_errors.txt 에 기록 (어느 필드가 어떻게 틀렸는지)
4. 위반 없으면 schema_errors.txt 는 생성하지 않음

11. 추론 실행

vLLM 서버를 띄워 VLM(Qwen VL 3)를 실행하고 데이터를 전처리 및 입력해 결과를 추출한다.

11-1. GPU 확인

```
nvidia-smi --query-gpu=memory.used,memory.free --format=csv,noheader
```

19 MiB, 40308 MiB (A100 40GB 기준) 인지 확인해 GPU 가 비어있는지 확인한다. 다른 프로세스가 점유 중이면 정리한다.(GPU 정리 방법은 13-3)

11-2. vLLM 서버 띄우기

서버 실행:

```
cd /home/jovyan/vlm_poc
mkdir -p logs
conda activate vllm

VLLM_BIN=/home/jovyan/miniconda3/envs/vllm/bin/vllm
MODEL=Qwen/Qwen3-VL-8B-Instruct

nohup $VLLM_BIN serve $MODEL \
    --gpu-memory-utilization 0.75 \
    --port 8000 \
    --max-model-len 49152 \
    --limit-mm-per-prompt '{"image":30}' \
    --dtype bfloat16 \
    > logs/vllm_server.log 2>&1 &

echo "vLLM PID: $!"
```

11-3. 준비 대기

```
until curl -s -o /dev/null --max-time 2 http://localhost:8000/v1/models \
2>/dev/null; do sleep 4; echo "waiting..."; done && echo "READY"
```

READY 출력되면 서버 준비 완료.

```
(vllm) jovyan@cheetah-796f6f62696e3131-6uly1d-6cf846895c-2mnnd:~/vllm_poc$ until curl -s
-o /dev/null --max-time 2 http://localhost:8000/v1/models 2>/dev/null; do sleep 4; ech
o "waiting..."; done && echo "READY"
READY
```

11-4. 추출 실행

또 다른 새 터미널 → vllm_poc 환경:

```
cd /home/jovyan/vllm_poc
conda activate vllm_poc

python scripts/preprocess.py --inputs inputs/ --out-dir outputs/run_001/ --dpi
100

python scripts/extract.py --run-dir outputs/run_001/ --prompts-dir prompts/ --
schema schema/extraction_schema.json --model-name Qwen/Qwen3-VL-8B-Instruct
```

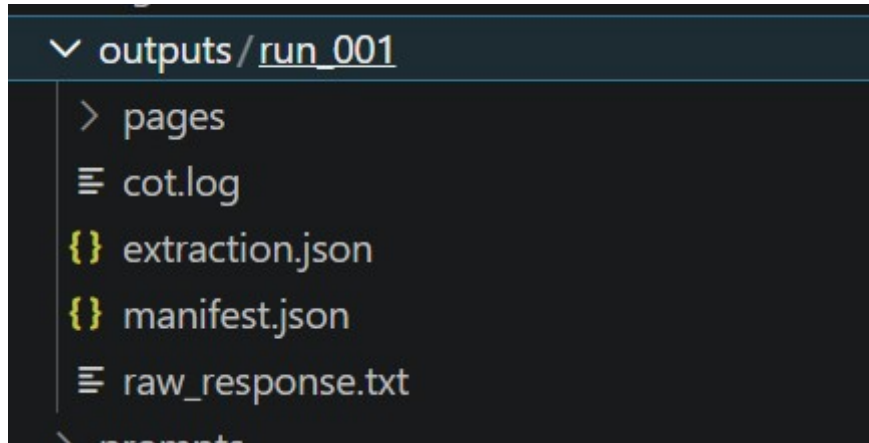
```
(vllm_poc) jovyan@cheetah-796f6f62696e3131-6uly1d-6cf846895c-2mnnd:~/vllm_poc$ python scr
ipts/preprocess.py --inputs inputs/ --out-dir outputs/run_001/ --dpi 100
[preprocess] wrote 26 page(s) → outputs/run_001/pages
[preprocess] manifest → outputs/run_001/manifest.json
```

```
(vllm_poc) jovyan@cheetah-796f6f62696e3131-6uly1d-6cf846895c-2mnnd:~/vllm_poc$ python scr
ipts/extract.py --run-dir outputs/run_001/ --prompts-dir prompts/ --schema schema/extra
ction_schema.json --model-name Qwen/Qwen3-VL-8B-Instruct
[extract] vLLM endpoint : http://localhost:8000/v1
[extract] model          : Qwen/Qwen3-VL-8B-Instruct
[extract] encoding 26 image(s) as base64 ...
[extract] requesting (max_tokens=2048) ...
[extract] done in 25.2s
  extraction.json → outputs/run_001/extraction.json
    cot.log       → outputs/run_001/cot.log
  raw_response    → outputs/run_001/raw_response.txt
```

11-5. 결과

outputs/run_001/에 4 개 파일.

파일	내용
extraction.json	메인 산출물. 스키마 준수 7 개 필드 JSON.
cot.log	모델 추론 과정(왜 그 값을 골랐는지).
raw_response.txt	모델 원본 응답.
schema_errors.txt	스키마 위반(있을 때만 생성).



extraction.json 과 cot.log 가 핵심 산출물이다. VSCode 사이드바에서 outputs/run_001/ 폴더를 열어 각 파일을 다음과 같이 읽는다.

extraction.json — 7 개 필드가 각각 하나의 객체로 들어 있다. 객체마다 value(추출한 값), source(근거가 된 파일·페이지·섹션), confidence(0~1 확신도)를 담고, 인화점·끊는점 같은 수치 필드는 unit·method 도 함께 갖는다. 한 필드를 읽을 때는 value 만 보지 말고 source 로 어느 문서 몇 페이지에서 나온 값인지, confidence 로 모델이 얼마나 확신하는지를 함께 본다. value 가 null 이거나 confidence 가 낮으면(예: 0.3 이하) 모델이 문서에서 근거를 찾지 못했다는 뜻이므로 사람이 원본을 다시 확인한다. ghs_classification 은 단일 값이 아니라 pictograms·signal_word·h_codes 로 나뉜 객체다.

cot.log — 모델이 출력한 <thinking> 블록만 따로 저장한 파일이다. 항목별로 번호를 매겨 어느 첨부의 어느 섹션을 보고 그 값을 골랐는지, 여러 문서에서 같은 값이 확인됐는지를 적는다.

extraction.json 을 먼저 보고 의심스러운 값이 있으면 cot.log 에서 그 항목의 추론을 찾아 근거 문서를 되짚는 식으로 읽는다. 문서에 없는 값을 모델이 지어냈는지(환각) 여부도 여기서 드러난다.

raw_response.txt — 모델이 내놓은 가공 전 응답 전체다. extraction.json 이나 cot.log 가 비어 있거나 이상할 때 원본 응답과 대조해 어디서 어긋났는지 확인하는 용도다.

11-6. vLLM 종료 (다음 작업 전 필수)

학습 같은 다음 작업을 하려면 vLLM 서버를 반드시 종료해 GPU 를 비워야 한다.

```
ps -eo pid,command | grep -iE "vllm|engine[_]?core|multiprocessing.resource_tracker" | grep -v grep | awk '{print $1}' | xargs -r kill -9 2>/dev/null; \n\nnvidia-smi --query=compute-apps=pid --format=csv,noheader | xargs -r kill -9 2>/dev/null; \
```

```
sleep 5; \  
nvidia-smi --query-gpu=memory.used,memory.free --format=csv,noheader
```

마지막 출력이 19 MiB 정도면 회수 완료.

12. 파인튜닝

파인튜닝은 모델을 자신의 화학물질·문서 양식에 맞추는 과정이다. 이 장은 크게 두 부분으로 나뉜다 — 7장에서 수집한 문서를 학습 데이터로 가공하는 **준비 단계(12-1 ~ 12-7)** 와, 실제로 LoRA 학습을 실행하는 **실행 단계(12-8 이후)** 다.

학습 구성은 QLoRA + LoRA + vision 인코더 freeze + gradient checkpointing 조합이다.

12-1. 학습 데이터 수집 — 다양성 확보

추론 입력만 모은다면 대상 문서를 그대로 모으면 된다. 그러나 파인튜닝 학습 데이터로 쓸 자료라면 다양성 을 의식하고 모아야 한다. 한쪽으로 치우친 데이터는 모델이 특정 패턴만 외우게 만든다.

본격적인 PoC 학습의 출발점은 약 1000 건 규모다.

다양성 항목	왜 / 어떻게
GHS 픽토그램	GHS01~GHS09 를 고르게 포함한다. 특정 픽토그램만 많으면 나머지를 학습하지 못한다.
벤더·레이아웃	여러 제조사·여러 문서 양식을 섞는다. 한 가지 레이아웃만 모으면 모델이 레이아웃 자체를 외운다.
언어	한국어 MSDS 와 영문 SDS 를 함께 넣는다.
엣지 케이스	인화점이 없는 물질(황산, 과산화수소 등)을 의도적으로 포함한다 — "없으면 null" 을 학습시킨다.
실제 라벨	빛 반사·기울기 노이즈가 있는 실제 라벨 사진을 섞어야 추론 강건성이 올라간다.

여기서 '**1 건**'은 **서로 다른 화학물질 하나**를 가리킨다. 이 파이프라인은 화학물질 하나당 train_set 샘플 하나(train_set/<chem>.json)를 만들기 때문에, 약 1000 건은 곧 **서로 다른 화학물질 약 1000 종**을 뜻한다.

같은 화학물질을 이미지 각도·자르기나 프롬프트만 바꿔 여러 건으로 부풀리는 데이터 증강 (augmentation)은 데모 스크립트에 들어 있지 않다. 다양성은 같은 물질을 반복해서가 아니라, 위 표처럼 서로 다른 물질·벤더·레이아웃·언어를 늘려서 확보한다.

한 화학물질은 보통 라벨 + 한국어 MSDS + 영문 SDS 약 3 개 문서로 이뤄지므로, 1000 종이면 원본 문서는 대략 3000 개, 페이지 이미지는 그보다 많아진다. 처음부터 1000 종을 채우기 어렵다면

픽토그램 9 종과 주요 벤더가 골고루 들어간 수십 종 규모로 시작해 점진적으로 늘려도 된다.

12-2. 학습 데이터 준비 — 수집한 자료에 무엇을 더하나

7 장에서 모은 것은 원본 문서(라벨 사진·MSDS·SDS PDF)다. 학습기(finetune_lora_v2.py)는 이 원본을 그대로 읽지 못한다. 원본에 다음 세 가지를 더해야 학습 데이터가 된다.

- **페이지 이미지** — PDF 를 페이지별 PNG 로 변환한다 (12-3).
- **정답 라벨** — 각 문서에서 추출되어야 할 7 개 항목의 정답을 사람이 작성한다 (12-5).
- **조립** — 이미지·프롬프트·정답을 학습기가 읽는 한 파일로 합친다 (12-7).

정리된 데이터는 9 장에서 본 ft_data_pack 과 동일한 폴더 구조를 갖는다.

```
data/ft_data_pack/  
├─ inputs/           7 장에서 수집한 원본 PDF·JPG  
├─ pages/<chem>/     페이지 PNG + manifest.json  
├─ ground_truth/     정답 JSON (<chem>.json)  
└─ train_set/        학습기 입력 (<chem>.json)
```

핵심 작업인 정답 라벨링과 thinking 작성은 사람이 직접 한다.

학습셋과 평가셋 나누기

모은 화학물질 전부를 학습에 쓰면, 모델이 정말 배웠는지 아니면 그 데이터를 외웠는지 구분할 수 없다. 그래서 일부 화학물질을 학습에서 빼 **평가셋(eval set)**으로 따로 둔다.

- **비율** — 전체의 약 10~20%를 평가용으로 뺀다. 의미 있는 수치를 보려면 최소 10 종 안팎은 확보하는 게 좋다.
- **구성** — 평가셋도 픽토그램·벤더·언어·엣지 케이스가 골고루 들어가야 한다. 쉬운 것만 평가하면 정확도가 부풀려진다.
- **정답은 만들되 학습엔 넣지 않는다** — 평가셋 화학물질도 ground_truth 는 똑같이 작성한다 (채점 기준이 된다). 단 train_set 에는 넣지 않는다.

평가셋은 data/eval_set/ 아래에 학습 데이터와 같은 구조로 둔다.

```
data/eval_set/  
├─ inputs/           평가용 원본 (학습에 안 쓴 화학물질)  
├─ pages/           페이지 PNG  
└─ ground_truth/     평가용 정답 JSON (<chem>.json)
```

data/eval_set/ground_truth_template.json 을 빈 양식으로 참고한다. 평가셋으로 실제 정확도를 재는 방법은 12-12 에서 다룬다.

12-3. 페이지 이미지 변환

inputs/ 에 화학물질별 원본을 둔 뒤, PDF·JPG 를 DPI 100 PNG 로 변환하고 화학물질별

manifest.json 을 생성한다. 추론 파이프라인의 preprocess.py 와 같은 동작이며, 페이지 순서는 label → 한국어 MSDS → 영문 SDS 로 정렬한다.

- 데모의 rasterize.py 는 5 종 (acetone·methanol·sulfuric_acid·hydrogen_peroxide·toluene)이 코드에 하드코딩되어 있다. 자신의 화학물질을 추가하려면 스크립트의 화학물질 키 목록을 수정하거나, 화학물질 단위로 변환을 돌린다.

결과물은 pages/<chem>/*.png 와 pages/<chem>/manifest.json 이다.

12-4. 프롬프트 구성

프롬프트는 prompts/system_prompt.txt 와 prompts/user_prompt_template.txt 두 파일로 구성된다. 추론이든 학습이든 이 둘을 이어붙인 하나의 프롬프트를 모델에 준다.

system_prompt.txt — 규칙과 출력 형식

모델의 역할, 반드시 지켜야 할 6 개 CORE RULES, 출력 골격(OUTPUT SHAPE)을 정의한다. 6 개 규칙은 다음과 같다.

- **EVIDENCE-ONLY** — 이미지에 인쇄되어 있거나 표·픽토그램에서 직접 읽히는 값만 쓴다. 외부 지식으로 빈칸을 채우지 않는다. 안 보이면 value 는 null, confidence 는 0.3 이하.
- **STRUCTURED OUTPUT** — 첨부된 JSON Schema 의 키 이름·required·enum 을 글자 그대로 따른다. 스키마에 없는 키를 만들지 않는다. JSON 외의 텍스트(설명·인사·마크다운 코드펜스)는 출력하지 않는다(<thinking> 만 예외).
- **SOURCE & CONFIDENCE** — 모든 값에 source(filename·page·section)와 confidence(0~1)를 단다. 둘 이상의 출처에서 같은 값이 확인되면 source.corroborated_by 에 추가 출처를 기록하고 confidence 를 0.95~0.99 로 올린다.
- **UNITS & FORMATS** — 온도는 °C, CAS 번호는 하이픈 표기, GHS 픽토그램은 GHS01~GHS09 코드, 응급조치는 한 문장으로 압축한다.
- **CHAIN-OF-THOUGHT** — JSON 앞에 <thinking>...</thinking> 블록을 두고, 항목마다 [N] 번 첨부된 어느 섹션·표·픽토그램을 봤는지 1~2 문장 적는다.
- **NO OUTSIDE KNOWLEDGE & SCOPE** — manifest 의 kind 같은 안내는 출처로 인정하지 않는다(이미지에서 다시 확인돼야 함). extraction_metadata 의 실제 값은 추론 후 시스템이 후처리로 채운다.

user_prompt_template.txt — 추출 지시

입력 묶음(MSDS·라벨·TDS)을 설명하고, 추출할 7 개 항목을 지시하는 부분이다. 두 곳에 자리표시자가 있다.

- **{file_manifest}** — 첨부된 이미지의 파일명·페이지를 첨부 순서대로 나열하는 자리.
- **{schema_json}** — extraction_schema.json 전문이 통째로 주입되는 자리. JSON 의 키·required·enum 은 이 스키마가 단일 진실이다.

파인튜닝에서 프롬프트를 다루는 법

- **프롬프트는 화학물질마다 새로 쓰지 않는다.** system_prompt.txt 와 user_prompt_template.txt 를 그대로 이어붙여 train_set 각 샘플의 prompt 로 쓴다.
- **샘플마다 바뀌는 것은 {file_manifest} 한 곳뿐이다** — 그 샘플에 들어가는 이미지의 파일명·페이지를 첨부 순서대로 채운다. {schema_json} 자리에는 extraction_schema.json 전체가 들어가며 모든 샘플에 동일하다.
- **추론과 학습이 같은 프롬프트를 써야 한다.** 프롬프트를 수정하면 추론 경로(11 장)도 같은 프롬프트로 맞춰야 모델이 일관되게 동작한다.
- **정답(response)의 형식은 system_prompt 의 OUTPUT SHAPE 와 글자 단위로 일치** 해야 한다 — <thinking> 블록 + JSON 객체 순서, 키 구조까지. 라벨러가 형식을 다르게 쓰면 모델이 잘못된 형식을 학습한다.

12-5. 정답 데이터 라벨링

화학물질마다 ground_truth/<chem>.json 을 하나씩 만들고, extraction_schema.json 형식 그대로 7 개 항목을 사람이 채운다. 라벨러는 문서를 직접 읽고 다음 규칙을 지킨다.

- **EVIDENCE-ONLY** — 문서에 인쇄되어 있거나 표·픽토그램에서 직접 읽을 수 있는 값만 적는다. 외부 지식으로 빈칸을 채우지 않는다. 보이지 않으면 value 는 null, confidence 는 0.3 이하.
- **출처와 신뢰도** — 모든 값에 source(filename, page, section)와 confidence(0~1)를 적는다. 두 개 이상의 문서에서 같은 값이 확인되면 source.corroborated_by 에 추가 출처를 기록하고 confidence 를 0.95~0.99 로 올린다.
- **인화점 없는 물질** — 인화점이 존재하지 않는 물질은 value·unit·method 를 모두 null, confidence 를 0.0 으로 명시한다. 임의의 숫자를 넣지 않는다.
- **단위 표기 통일** — NumericField 의 unit 표기는 system_prompt 가 지시하는 표기와 똑같이 맞춘다. 데모 데이터는 "celsius" 로 되어 있고 system_prompt 는 °C 를 지시해 서로 어긋나므로, 자신의 데이터는 한쪽으로 통일한다.
- **extraction_metadata** — 정답 JSON 에서는 { "value": null, "confidence": 0.0 } 으로 둔다(추론 시 시스템이 채운다).

아래는 실제 예시다 — methanol 의 정답 JSON 중 세 필드. 문자열 항목(product_name), 객체 항목(ghs_classification), 수치 항목(flash_point_celsius)의 형태를 보여준다.

```

{
  "product_name": {
    "value": "METHANOL",
    "source": { "filename": "methanol_label.jpg", "page": 1,
                 "section": "product_name" },
    "confidence": 0.97
  },
  "ghs_classification": {
    "pictograms": ["GHS02", "GHS06", "GHS08"],
    "signal_word": "Danger",
    "h_codes": ["H225", "H301", "H311", "H331", "H370"],
    "source": { "filename": "methanol_label.jpg", "page": 1,
                 "section": "ghs_classification" },
    "confidence": 0.96
  },
  "flash_point_celsius": {
    "value": 11.0,
    "unit": "celsius",
    "method": "closed cup",
    "source": { "filename": "methanol_sds_en.pdf", "page": 3,
                 "section": "9.1 Information on basic physical and chemical
properties" },
    "confidence": 0.92
  }
}

```

나머지 manufacturer·boiling_point_celsius·first_aid_measures 도 같은 구조다. 전체 형식은 ft_data_pack/ground_truth/methanol.json 과 data/eval_set/ground_truth_template.json 을 참고한다.

```

1  {
2    "_comment": "이 파일은 평가 샘플 한 건의 ground_truth.json 템플릿입니다. 라벨러는 이 구조를
3
4    "product_name": {
5      "value": "Acetone",
6      "source": {
7        "filename": "acetone_msds_ko.pdf",
8        "page": 1,
9        "section": "1",
10       "corroborated_by": [
11         { "filename": "acetone_label.jpg", "page": 1, "section": "label_front" }
12       ]
13     },
14     "confidence": 0.99
15   },
16
17   "manufacturer": {
18     "value": "에스케이케미칼대정주식회사",
19     "source": {
20       "filename": "acetone_msds_ko.pdf",
21       "page": 1,
22       "section": "1"
23     },
24     "confidence": 0.95
25   },
26
27   "cas_number": {
28     "value": "67-64-1",
29     "source": {
30       "filename": "acetone_msds_ko.pdf",
31       "page": 1,
32       "section": "1",
33       "corroborated_by": [
34         { "filename": "acetone_label.jpg", "page": 1, "section": "label_front" }

```

12-6. 정답 항목별 작성법

정답 라벨링의 일반 규칙(EVIDENCE-ONLY, 출처·신뢰도 표기 등)은 12-5 를 따른다. 이 절은 **7 개 항목 각각을 무엇으로·어디서·어떤 형식으로 채우는지** 를 구체적으로 다룬다.

항목 공통 구조

정답은 7 개 항목으로 이뤄진 하나의 JSON 객체다. 각 항목은 다시 객체이며, 타입은 세 가지다.

- **StringField** — value · source · confidence (product_name · manufacturer · cas_number).
- **NumericField** — value · unit · method · source · confidence (flash_point_celsius · boiling_point_celsius).
- **객체형** — 항목 고유의 키 묶음 + source + confidence (ghs_classification · first_aid_measures).

각 타입의 골격은 다음과 같다.

```
StringField : { "value": <문자열 | null>, "source": {...}, "confidence": <0~1> }

NumericField : { "value": <숫자 | null>, "unit": "°C", "method": <문자열 | null>,
                  "source": {...}, "confidence": <0~1> }

source      : { "filename": "<파일명>", "page": <정수>, "section": "<섹션>",
                  "corroborated_by": [ { "filename": ..., "page": ... } ] }
```

source 의 키는 file 이 아니라 **filename** 이며 filename·page 는 필수다. confidence 는 0~1 숫자로, 단일 출처는 0.85~0.95, 둘 이상 출처가 일치하면 0.95~0.99, 흐릿하거나 일부만 보이면 0.5~0.7, 안 보이면 0.0~0.3 으로 적는다.

항목별 작성법

항목	작성법
product_name (StringField)	제품의 공식 명칭. 라벨 상단, MSDS 1 번 항목. 영문·한글 중 가장 두드러진 표기 하나. 예: METHANOL
manufacturer (StringField)	제조사 또는 공급자 회사명. 라벨 공급자란, MSDS 1 번 항목. 회사명 전체 그대로. 예: SAMCHUN PURE CHEMICAL CO., LTD.
cas_number (StringField)	CAS 등록번호. 라벨, MSDS 3 번 항목(구성성분). 'NN-NN-N' ~ 'NNNNN-NN-N' 하이픈 표기 그대로. 양 끝 공백·괄호 제거. 예: 67-56-1
ghs_classification (객체)	GHS 픽토그램·신호어·H 문구. 라벨, MSDS 2 번 항목(유해성·위험성). pictograms: GHS01~GHS09 코드 배열 / signal_word: Danger·Warning·위험·경고 또는 null / h_codes: H+세 자리 코드 배열. 픽토그램 명칭만 보이면 코드로 매핑(Flame→GHS02 등).
flash_point_celsius (NumericField)	인화점. MSDS 9 번 항목(물리·화학적 특성), 라벨 물성표. value 숫자, unit "°C", method 에 측정법(예: closed cup). 화씨로만 있으면 °C 로 변환하고 method 에 'converted from °F'. 인화점이 없는 물질은 value·unit·method 모두 null, confidence 0.0.
boiling_point_celsius (NumericField)	끓는점. MSDS 9 번 항목. value 숫자, unit "°C", method 에 압력 조건(예: 1013 hPa).
first_aid_measures (객체)	응급조치 요령. MSDS 4 번 항목. inhalation·skin_contact·eye_contact·ingestion 네 경로별 각 한 문장. 원문을 그대로 옮기지 말고 핵심 행동만 압축.

항목	작성법
	해당 경로 정보가 없으면 그 키만 null.

extraction_metadata (8 번째 항목)

extraction_metadata 는 스키마의 8 번째 항목이지만 추출 내용이 아니라 메타데이터(사용 모델 · 시각 · 입력 파일 목록)다. 이 값은 추론 후 시스템이 후처리로 채우므로 **라벨러는 작성하지 않는다**.
데모 train_set 의 정답 JSON 에는 이 자리에 placeholder 로 { "value": null, "confidence": 0.0 } 이 들어 있다.

작성 예시 — first_aid_measures

객체형 항목의 예시다(methanol). 네 경로 각각을 한 문장으로 압축하고, 객체 전체에 하나의 source · confidence 를 단다.

```
"first_aid_measures": {
  "inhalation": "Move to fresh air. If not breathing, give artificial respiration. Consult a physician immediately.",
  "skin_contact": "Take off contaminated clothing immediately. Wash off with soap and plenty of water. Consult a physician.",
  "eye_contact": "Rinse thoroughly with plenty of water for at least 15 minutes and consult an ophthalmologist.",
  "ingestion": "Do NOT induce vomiting. Rinse mouth with water. Call a physician immediately.",
  "source": { "filename": "methanol_sds_en.pdf", "page": 2, "section": "4.1 Description of first aid measures" },
  "confidence": 0.95
}
```

문자열 · 수치 항목 예시와 전체 7 개 항목 형식은 12-5 의 methanol 예시, ft_data_pack/ground_truth/methanol.json, data/eval_set/ground_truth_template.json 을 함께 참고한다.

12-7. train_set 조립

정답 · 페이지 이미지 · 프롬프트를 학습기가 읽는 형식으로 합친다. 화학물질마다 train_set/<chem>.json 을 하나씩 만들며, image_paths · prompt · response 세 키로 구성된다.

```
{
  "image_paths": [
    "pages/methanol/methanol_label.png",
    "pages/methanol/methanol_msds_ko_p001.png",
    "...",
    "pages/methanol/methanol_sds_en_p004.png"
  ],
  "prompt": "<system_prompt 전문>\n\n<user_prompt_template 전문>",
  "response": "<thinking>...</thinking>\n{ ...7 필드 JSON... }"
}
```

- **image_paths** — ft_data_pack 을 기준으로 한 상대경로를 label → 한국어 MSDS → 영문 SDS 순으로 나열한다. 학습 시 finetune_lora_v2.py 의 --image-root 옵션이 절대경로로 해석한다(12-9).
- **prompt** — prompts/system_prompt.txt 와 prompts/user_prompt_template.txt 를 그대로 이어붙인 하나의 문자열이다.
- **response** — <thinking> 블록 + 7 필드 JSON. JSON 은 12-5 의 ground_truth/<chem>.json 내용을 그대로 쓴다.

<thinking> 블록은 화학물질마다 사람이 작성한다. [N]번 페이지 인덱스로 어느 자료의 어느 섹션을 보았는지, 여러 문서가 일치하는지를 항목마다 적는다. 아래는 methanol 의 실제 <thinking> 발췌다.

```
<thinking>
1. 입력 자료를 식별: 라벨 이미지 1 장 [1], 한국어 MSDS PDF 4 페이지 [2-5],
   영문 SDS PDF 4 페이지 [6-9]. 모두 동일 화학제품 METHANOL (CAS 67-56-1).

2. 제품명: 라벨 [1] 상단에 "METHANOL", 한국어 MSDS [2] 1.가 항목에
   "메탄올 (메틸알코올)", 영문 SDS [6] 1.1 Product name 에 동일 표기.
   영문명을 canonical 값으로 채택, source 는 라벨로 지정.

   ( ... 3. 제조사, 4. GHS 분류 항목 동일 형식으로 ... )

5. 인화점: 라벨 [1] Key Physical Data 에 11.0 °C (closed cup), 영문 SDS
   [8] Section 9.1 g) Flash point 에서 동일 확인. 두 출처 일치 →
   SDS 9.1 을 1차 출처로 채택, unit="celsius" 명시.
</thinking>
```

핵심 규칙 — response 의 <thinking> 형식과 JSON 구조는 system_prompt 가 모델에게 요구하는 출력 형식과 글자 단위로 일치해야 한다. 라벨러가 형식을 다르게 쓰면 모델은 잘못된 형식을 학습한다.

train_set 을 다 만들었으면 학습 전에 sanity check 를 돌린다.

```
cd data/ft_data_pack
python <<'PY'
import json, glob, re
for p in sorted(glob.glob("train_set/*.json")):
    s = json.load(open(p))
    m = re.search(r"</thinking>\s*(\{.*\})\s*$", s["response"], re.S)
    assert m, f"thinking/JSON 분리 실패: {p}"
    json.loads(m.group(1))
    print("OK", p, len(s["image_paths"]), "pages")
PY
```

12-8. GPU 비우기

학습 실행 단계로 들어간다. 11-6 절차로 vLLM 서버를 완전히 종료해 GPU 를 회수한 후:

```
conda activate vlm_poc
cd /home/jovyan/vlm_poc
mkdir -p logs checkpoints
```

12-9. 학습 실행

```
python scripts/finetune_lora_v2.py \
  --train-dir data/ft_data_pack/train_set \
  --image-root data/ft_data_pack \
  --output-dir checkpoints/run_a/ \
  --model-name Qwen/Qwen3-VL-8B-Instruct \
  --precision bf16 \
  --qlora \
  --freeze-vision-encoder \
  --target-modules llm-linear \
  --per-device-batch-size 1 \
  --grad-accum 4 \
  --lr 2e-4 \
  --rank 16 \
  --alpha 32 \
  --epochs 3 \
  --gradient-checkpointing \
  2>&1 | tee logs/lora_run.log
```

--image-root data/ft_data_pack 는 train_set 의 상대경로 image_paths 를 해석할 기준 디렉터리다. finetune_lora_v2.py 가 학습 시점에 각 경로를 이 디렉터리 기준 절대경로로 바꿔 이미지를 읽는다. **이 옵션을 빼면 상대경로가 현재 작업 디렉터리 기준으로 해석되어 이미지 파일을 찾지 못하고 학습이 실패한다.**

--qlora, --freeze-vision-encoder, --gradient-checkpointing 조합은 GPU 메모리를 최대한 절약하는 구성이다. --target-modules llm-linear 는 어텐션뿐 아니라 LLM 의 MLP linear 까지 LoRA 학습 대상에 포함한다는 의미다. 학습이 끝나면 checkpoints/run_a/final/adapter_model.safetensors 가 생성된다(크기는 rank·target-modules 에 따라 다르므로 실제 산출물로 확인).

12-10. 어댑터 머지

학습된 LoRA 어댑터를 base 모델에 합쳐 단독 모델을 생성한다.

```
python scripts/merge_lora.py --base-model Qwen/Qwen3-VL-8B-Instruct \
  --adapter-dir checkpoints/run_a/final \
  --output-dir checkpoints/run_a/merged
```

12-11. 학습 모델로 재추론

이 단계는 8B 모델 추론이므로 A100 40GB 한 장으로도 수행할 수 있다. 11 장 절차를 MODEL 경로만 머지 모델로 바꿔 반복한다.

```
conda activate vllm

VLLM_BIN=/home/jovyan/miniconda3/envs/vllm/bin/vllm
MODEL=/home/jovyan/vlm_poc/checkpoints/run_a/merged

cd /home/jovyan/vlm_poc
nohup $VLLM_BIN serve $MODEL \
  --gpu-memory-utilization 0.75 \
  --port 8000 \
  --max-model-len 49152 \
  --limit-mm-per-prompt '{"image":30}' \
  --dtype bfloat16 \
  > logs/vllm_finetuned.log 2>&1 &
```

11-3 의 READY 대기 절차를 거친 뒤, 11-4 와 동일하게 extract 를 호출한다(단 --model-name 도 머지 경로로 변경).

```
conda activate vlm_poc
cd /home/jovyan/vlm_poc

python scripts/preprocess.py --inputs inputs/ --out-dir outputs/run_a/ --dpi 100

python scripts/extract.py --run-dir outputs/run_a/ \
  --prompts-dir prompts/ \
  --schema schema/extraction_schema.json \
  --model-name /home/jovyan/vlm_poc/checkpoints/run_a/merged
```

12-12. 학습 전후 비교

기본 모델 추론 결과(run_001)와 학습 모델 추론 결과(run_a)를 비교한다.

```
diff outputs/run_001/extraction.json outputs/run_a/extraction.json
```

diff 는 두 출력이 어디서 달라졌는지 눈으로 보여줄 뿐, 정답과 얼마나 맞는지는 알려주지 않는다. 실제 정확도는 12-2 에서 떼어 둔 평가셋으로 잰다. 학습 모델을 평가셋 입력에 대해 11 장과 같은 방식으로(단 inputs 를 eval_set 으로 지정해) 한 번 더 추론시킨 뒤, 그 출력을 평가셋 정답과 항목별로 비교한다.

아래는 평가셋의 한 화합물질(methanol)을 추론해 outputs/eval_methanol/extraction.json 을 얻었다고 할 때, 그 결과를 평가셋 정답과 대조해 항목별 정답률을 내는 예시다.

```
python <<'PY'
import json
FIELDS = ["product_name", "manufacturer", "cas_number", "ghs_classification",
          "flash_point_celsius", "boiling_point_celsius", "first_aid_measures"]
```

```

pred = json.load(open("outputs/eval_methanol/extraction.json", encoding="utf-8"))
gold = json.load(open("data/eval_set/ground_truth/methanol.json", encoding="utf-8"))

def value_only(field, obj):
    # source.confidence 는 빼고 추출된 값만 비교한다
    item = obj.get(field, {})
    return {k: v for k, v in item.items() if k not in ("source", "confidence")}

hit = 0
for f in FIELDS:
    ok = value_only(f, pred) == value_only(f, gold)
    hit += int(ok)
    print(f"'0' if ok else 'X' {f}")
print(f"\n항목 정답률: {hit}/{len(FIELDS)} = {hit / len(FIELDS) * 100:.0f}%")
PY

```

여러 화학물질이면 같은 비교를 반복해 전체 평균 정답률을 낸다(위는 값이 완전히 같아야 정답으로 치는 엄격한 기준이며, 필요하면 항목별로 느슨하게 바꾼다). **학습 후 정답률이 학습 전보다 올라갔는지가 파인튜닝의 실제 성과다.**

13. 트러블슈팅

자주 만나는 케이스. 표에 없는 오류 메시지는 그대로 복사해 Claude Code 에 문의

13-1. 환경·실행

증상	원인 / 해결
ModuleNotFoundError: No module named '<X>'	conda activate 누락 또는 패키지 미설치. conda activate vlm_poc 후에도 안 되면 pip install <X>. 자주 누락: datasets, trl, openai, peft.
FileNotFoundError: 'pdfinfo'	PATH 문제. conda activate vlm_poc 후 실행. 절대 경로로 python 호출하면 PATH 에 conda bin 이 안 들어가서 실패.
[extract] missing page image referenced by manifest	extract.py 의 image path 결함 버그. 현재 패키지에는 수정 반영됨.

13-2. vLLM 서버

증상	원인 / 해결
5 분 이상 준비 안 됨	tail -50 logs/vllm_server.log 로 원인 파악. CUDA OOM 이면 GPU 점유 확인. Loading weights 에서 멈춤이면 디스크/HF 캐시 부족(df -h ~).
BadRequestError: max_tokens must be at least 1, got -N	prompt 토큰 수가 --max-model-len 초과. 즉시 해결은 --max-model-len 49152 로 늘리기. 근본 해결은 preprocess.py 에 --dpi 100.

13-3. GPU 메모리 회수 안 됨

nvidia-smi 에 점유 표시 남아 있으면 자식 프로세스가 init 으로 reparent 된 상태. ps 로 모두 찾아서 kill.

```
ps -eo pid,command | grep -iE "vllm|engine[_]?core|multiprocessing.resource_tracker" | grep -v grep | awk '{print $1}' | xargs -r kill -9 2>/dev/null; \

nvidia-smi --query-compute-apps=pid --format=csv,noheader | xargs -r kill -9 2>/dev/null; \

sleep 5; \
```

```
nvidia-smi --query-gpu=memory.used,memory.free --format=csv,noheader
```

13-4. 학습 관련

증상	원인 / 해결
IndexError: image_grid_thw[N] out of bounds	datasets.Dataset.from_list().map() 패턴이 multimodal 처리를 깨뜨림. starter finetune_lora.py 말고 finetune_lora_v2.py 사용.
CUDA OOM	순서대로 시도: --grad-accum 1 → --max-samples 2 → acetone 페이지 4 장으로 축소 → liger-kernel 설치 → H100/multi-GPU.
progress 0%에서 멈춤, GPU 사용률 0	kernel 버전 문제. "Detected kernel version 5.4.0" 경고가 보이면 finetune_lora_v2.py 의 SFTConfig 에서 dataloader_num_workers=0 으로.

13-5. 다른 사용자가 GPU 점유 중

```
nvidia-smi --query-compute-apps=pid,process_name,used_memory --format=csv
```

본인 프로세스 아니면 운영팀 문의.

14. 부록

14-1. 명령어 치트시트

환경 활성화

```
conda activate vlm_poc      # 추론 클라이언트 + 학습
conda activate vllm         # vLLM 서버
```

vLLM 시작·확인·종료

```
VLLM_BIN=/home/jovyan/miniconda3/envs/vllm/bin/vllm
MODEL=Qwen/Qwen3-VL-8B-Instruct

nohup $VLLM_BIN serve $MODEL --gpu-memory-utilization 0.75 --port 8000 \
    --max-model-len 49152 --limit-mm-per-prompt '{"image":30}' \
    --dtype bfloat16 > logs/vllm_server.log 2>&1 &

curl -s http://localhost:8000/v1/models | python -m json.tool

ps -eo pid,ppid,command | grep -iE "vllm|VLLM" | grep -v grep
kill -9 <pids>
```

추론

```
python scripts/preprocess.py --inputs inputs/ --out-dir outputs/<run>/ --dpi
100
python scripts/extract.py --run-dir outputs/<run>/ --prompts-dir prompts/ \
    --schema schema/extraction_schema.json \
    --model-name Qwen/Qwen3-VL-8B-Instruct
```

파인튜닝 + 머지

```
python scripts/finetune_lora_v2.py \
    --train-dir data/ft_data_pack/train_set \
    --image-root data/ft_data_pack \
    --output-dir checkpoints/<run>/ \
    --model-name Qwen/Qwen3-VL-8B-Instruct \
    --precision bf16 --qlora --freeze-vision-encoder \
    --target-modules llm-linear \
    --per-device-batch-size 1 --grad-accum 4 --lr 2e-4 \
    --rank 16 --alpha 32 --epochs 3 --gradient-checkpointing \
    2>&1 | tee logs/lora_run.log

python scripts/merge_lora.py --base-model Qwen/Qwen3-VL-8B-Instruct \
    --adapter-dir checkpoints/<run>/final --output-dir
checkpoints/<run>/merged
```

15. 부록 — Qwen3.5-9B 대체 백본 (추론 비교)

기본 PoC 는 본문 5 장의 Qwen3-VL-8B-Instruct 를 그대로 쓴다. 이 부록은 비교·대체 평가를 위해 Qwen3.5-9B 를 병렬로 구성하는 방법이다. 기존 환경·모델·추론 결과(run_001)는 그대로 두고 추가만 한다. 현재 검증된 범위는 추론·비교까지이며(2026-06-11 전체 시스템 E2E 통과), 파인튜닝(12 장)은 Qwen3-VL 기준 그대로다. Qwen3.5 파인튜닝은 별도 검증 전까지 범위 밖이다.

15-1. 무엇이 다른가 (Qwen3-VL → Qwen3.5)

Qwen3.5 는 몇 가지 지점에서 기존 Qwen3-VL 과 다르고, 그 차이가 별도 env·플래그·토큰 예산을 요구한다. 핵심 차이는 다음과 같다.

항목	Qwen3-VL-8B (본문)	Qwen3.5-9B (이 부록)
모델	Qwen/Qwen3-VL-8B-Instruct	Qwen/Qwen3.5-9B (자체 멀티모달, -VL 변형 없음, Apache-2.0, ~19GB)
아키텍처	표준 VLM	Qwen3_5ForConditionalGeneration (Gated DeltaNet 하이브리드)
conda env	vllm (py3.11)	vllm35 (py3.12) — 신규 아키텍처 때문에 별도 env 필수. 기존 vllm 은 안 건드림.
reasoning	프롬프트 유도 <thinking>	reasoning 모델 — 네이티브 <think>...</think>. serve 시 --reasoning-parser qwen3.
serve gpu-util	0.75	0.85
extract 토큰	기본값(2048)로 충분	--max-new-tokens 16000 필수 (아래 15-4)
GPU 공존	—	A100 40GB 한 장 → 기존 Qwen3-VL 서버와 동시 불가

15-2. 환경 구축 — conda env vllm35

기존 vllm env(py3.11)는 Qwen3.5 의 신규 아키텍처를 지원하지 못하므로 별도 env 가 필요하다. 기존 env 는 건드리지 않는다. Python 3.12 로 새 env 를 만들고, vLLM 은 아래 +cu129 휠을 지정 설치한다(CUDA 함정 주의 — 바로 아래).

```
conda create -n vllm35 python=3.12 -y
```

```
conda activate vllm35
```

```
# vLLM 0.22.1 의 +cu129 릴리스 휠 (libcudart.so.12 = CUDA 12.9). nightly 금지 —  
아래 함정 참고
```

```
pip install
```

```
"https://github.com/vllm-project/vllm/releases/download/v0.22.1/vllm-  
0.22.1+cu129-cp38-abi3-manylinux_2_28_x86_64.whl"
```

```
# transformers main (5.10.0.dev0) — Qwen3.5 아키텍처 지원
```

```
pip install -U "git+https://github.com/huggingface/transformers"
```

→ 이 휠이 torch 2.11.0+cu129 까지 함께 끌고 온다. 바이너리 경로:

/home/jovyan/miniconda3/envs/vllm35/bin/{python,vllm}.

⚠ CUDA 함정 (반드시 이 휠로)

vLLM nightly 를 --torch-backend=auto 로 깔면 vLLM_C 가 libcudart.so.13(CUDA 13) 을 요구해, 이 박스의 드라이버 550(CUDA 12.4)에서 import 에 실패한다(vLLM 알려진 버그 #43435). 해결은 nightly 대신 안정 0.22.1 의 +cu129 휠(=libcudart.so.12)을 쓰는 것이다. cu128 휠은 0.22.1 에 없다.

15-3. 모델 다운로드

공유 HF 캐시(~/.cache/huggingface/hub/)에 받는다. hf CLI 는 기존 vlm_poc env 로 충분하다(6 장에서 설치·로그인 완료).

```
conda activate vlm_poc
```

```
hf download Qwen/Qwen3.5-9B # 약 19GB
```

15-4. 추론 실행

GPU 는 한 장이다. 기존 Qwen3-VL 서버가 떠 있으면 11-6 절차로 먼저 내려 GPU 를 비운다. 서버 기동은 inline 명령 대신 런치 스크립트를 쓴다.

```
cd ~/vlm_poc
```

```
./launch_qwen35.sh start # ready 폴링까지
```

```
./launch_qwen35.sh status
```

serve 기본값(스크립트 env 변수로 조정): port 8000, gpu-util 0.85, max-model-len 49152, limit image 30, dtype bf16, --reasoning-parser qwen3, --enforce-eager, VLLM_USE_FLASHINFER_SAMPLER=0. 네이티브 262K 컨텍스트는 KV 폭발이라 49152 로 캡했다. OOM 이면 더 낮추거나 gpu-util 조정.

추출은 기존 클라이언트를 그대로 쓰되, **모델명과 --max-new-tokens 16000 만 바꾼다.**

```
conda activate vlm_poc
python scripts/preprocess.py --inputs inputs/ --out-dir
outputs/run_002_qwen35/ --dpi 100
python scripts/extract.py --run-dir outputs/run_002_qwen35/ --prompts-dir
prompts/ \
    --schema schema/extraction_schema.json --model-name Qwen/Qwen3.5-9B --max-
new-tokens 16000
./launch_qwen35.sh stop
```

⚠️ --max-new-tokens 16000 은 선택이 아니라 필수

extract.py 기본값 --max-new-tokens 2048 로는 모델이 한국어 <thinking> 추론에 토큰을 다 써버려 최종 JSON 을 못 내놓고 "No JSON object found" 로 실패한다. 실측 completion 7,156 토큰. prompt 30K + 16K < max-model-len 49152 라 안전하다.

cot.log 동작 (검증됨)

프롬프트가 유도하는 <thinking> 블록은 message.content 로 나와 기존 extract.py 가 그대로 cot.log 에 기록한다(수정 불필요). --reasoning-parser qwen3 가 잡는 것은 네이티브 <think> 태그라 둘은 별개고, 모델은 <thinking> 를 내므로 reasoning_content 로 새지 않는다. 단, 프롬프트를 바꾸면 이 동작을 재확인해야 한다.

15-5. 트러블슈팅 (Qwen3.5 전용)

증상	원인 / 해결
import 실패: libcudart.so.13 요구 (vLLM_C)	CUDA 13 원인. nightly 대신 0.22.1 +cu129 휠 (libcudart.so.12) 설치. cu128 휠은 0.22.1 에 없음. (15-2 참고)
EngineCore 크래시: FileNotFoundError: ninja	FlashInfer 샘플러가 top-k/top-p 커널을 ninja+nvcc 로 JIT 빌드하려다 실패(이 박스엔 nvcc 없음). VLLM_USE_FLASHINFER_SAMPLER=0 으로 네이티브 샘플러 사용(greedy 라 품질 무관). 런치 스크립트가 export 함.
"No JSON object found" / 최종 JSON 미출력	reasoning 토큰 예산 부족. --max-new-tokens 16000 필요(기본 2048 부족). 15-4 참고.
모델 로드 자체 실패	--trust-remote-code 추가. transformers/vLLM 최신화 재확인.
너무 느림 (~212s)	--enforce-eager 때문. 검증 끝났으면 제거 시 CUDA graph/컴파일로 빨라짐(단 inductor 컴파일이 ninja/g++ 필요 — env/bin 이 PATH 에). 런치

	스크립트가 처리.
--	-----------

15-6. 검증 결과 (2026-06-11, run_002_qwen35)

전체 MSDS 파이프라인(preprocess 26 페이지 → extract)이 Qwen3.5-9B 로 정상 동작했다.

항목	결과
schema 위반	0 건 (run_001 의 의도적 9 건과 대비). filename / unit / corroborated_by 모두 정확.
추출 품질	전 항목 추출 + 라벨↔SDS 교차검증(corroborated_by) 작동, 한국어 CoT.
cot.log	flash_point -20°C(라벨) vs -16.99°C(SDS) 불일치, 제조사 불일치까지 추론.
usage	prompt 30,055 / completion 7,156 tok. elapsed 212.9s (enforce-eager).
run_001	그대로 보존됨.